

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 12 Использование C/C++ и Fortran

Язык Julia предоставляет возможности для взаимодействия с библиотеками, написанными на C/C++ и Fortran. Это особенно важно при решении сложных вычислительных задач, где требуется использовать высокооптимизированные внешние библиотеки.

Допустим, у нас есть скомпилированная библиотека `library.so`, содержащая функцию `function_name`. Если эта функция принимает аргументы `arg1`, `arg2`, ..., `argn` типов `arg_type1`, `arg_type2`, ..., `arg_typen` и возвращает результат типа `return_type`, то вызов этой функции в Julia выполняется следующим образом:

```
ccall((:function_name, "library.so"), return_type, (arg_type1,  
arg_type2, ..., arg_typen), arg1, arg2, ..., argn)
```

При использовании `ccall` для указания типов передаваемых данных применяются специальные типы с префиксом `C`, которые гарантируют точное соответствие с типами в языках C и Fortran.

- `Cint` - соответствует типу `int`
- `Cdouble` - соответствует типу `double`
- `Cfloat` - соответствует типу `float`
- `Cvoid` - соответствует типу `void`
- `Ptr{Cdouble}` - указатель на массив элементов типа `double`

Для библиотек, написанных на Fortran, сохраняется аналогичный синтаксис вызова, но есть важная особенность в именовании функций. Большинство компиляторов Fortran автоматически добавляют символ подчеркивания в

конце имен функций. Это означает, что функция, объявленная в исходном коде как `function_name`, в скомпилированной библиотеке будет доступна под именем `function_name_`.

Таким образом, при вызове Fortran-функций в Julia необходимо учитывать это преобразование имен и указывать имя функции с добавленным подчеркиванием.

Julia позволяет встраивать код на C/C++ и Fortran прямо в файлы с кодом на Julia благодаря поддержке вызова внешних функций. Для упрощения этого процесса можно разработать специализированный макрос, который автоматизирует работу с внешним кодом. В качестве примера ниже приведена авторская реализация такого макроса для C.

```
In [58]: macro C(str_expr)
    str = eval(str_expr) # Вычисляем выражение, переданное в макрос, чтобы
    # Создаем временные имена для файлов: исходного .c и библиотеки .so
    file = tempname() * ".c"
    lib = tempname() * ".so"
    # Записываем строку с кодом C во временный .c файл
    open(file, "w") do f
        write(f, str)
    end
    run(`gcc -shared -fPIC -o $lib $file`) # Компилируем код в библиотеку с
    return :(ccall(:main, $lib), Cvoid, ())
end
```

```
Out[58]: @C (macro with 4 methods)
```

Пример использования макроса:

```
In [59]: @C raw"""
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

void main() {
    printf("Привет из C!\n");
    int x = add(5, 7);
    printf("5 + 7 = %d\n", x);
}
"""
```

Привет из C!

5 + 7 = 12

```
In [60]: macro C(str_expr)
    str = eval(str_expr) # Вычисляем выражение, переданное в макрос, чтобы
    # Создаем временные имена для файлов: исходного .c и библиотеки .so
    file = tempname() * ".c"
    lib = tempname() * ".so"
    # Записываем строку с кодом C во временный .c файл
    open(file, "w") do f
        write(f, str)
    end
    run(`gcc -shared -fPIC -o $lib $file`) # Компилируем код в библиотеку с
    return :(ccall(:main, $lib), Cvoid, ())
end
```

```

lib = tempname() * ".so"
# Записываем строку с кодом C во временный .c файл
open(file, "w") do f
    write(f, str)
end
run(`g++ -shared -fPIC -o $lib $file`) # Компилируем код в библиотеку с
return :(ccall(:main, $lib), Cvoid, ())
end

@C raw"""
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    std::cout << "Привет из C++!" << std::endl;
    int x = add(5, 7);
    std::cout << "5 + 7 = " << x << std::endl;
    return 0;
}
"""

```

Привет из C++!

5 + 7 = 12

Макрос для Fortran:

```

In [32]: macro F(str_expr)
    str = eval(str_expr)
    file = tempname() * ".f90"
    lib = tempname() * ".so"
    open(file, "w") do f
        write(f, str)
    end
    run(`gfortran -shared -fPIC -o $lib $file`)
    return :(ccall(:main_, $lib), Cvoid, ())
end

```

Out[32]: @F (macro with 1 method)

Пример использования макроса:

```

In [40]: @F raw"""
real function multiply(x, y)
    implicit none
    real, intent(in) :: x, y
    multiply = x * y
end function multiply

real function main()
    implicit none
    real :: a, b, result
    real :: multiply

```

```

    print *, "Привет из Fortran!"
    a = 5.0
    b = 7.0
    result = multiply(a, b)

    print *, a, " * ", b, " = ", result
end function main
"""

```

```

Привет из Fortran!
  5.00000000      *    7.00000000      =    35.00000000

```

Макросы позволяют вызывать главную функцию без передачи параметров и обработки возвращаемых значений. Теперь реализуем код с передачей параметров в функцию и возвратом результата.

```

In [49]: macro C(str_expr)
          str = eval(str_expr)
          file = tempname() * ".c"
          lib = tempname() * ".so"

          open(file, "w") do f
            write(f, str)
          end

          run(`gcc -shared -fPIC -o $lib $file`)
          return lib
        end

# Сохраняем путь к библиотеке в глобальную константу
lib = @C raw""
int add_numbers(int a, int b) {
    return a + b;
}

int multiply_numbers(int a, int b) {
    return a * b;
}
"""

a = 5
b = 7

result1 = ccall((:add_numbers, lib), Cint, (Cint, Cint), a, b)
result2 = ccall((:multiply_numbers, lib), Cint, (Cint, Cint), a, b)

println("Первое число: ", a)
println("Второе число: ", b)
println("Сумма: ", result1)
println("Произведение: ", result2)

```

```

Первое число: 5
Второе число: 7
Сумма: 12
Произведение: 35

```

```
In [50]: macro F(str_expr)
    str = eval(str_expr)
    file = tempname() * ".f90"
    lib = tempname() * ".so"

    open(file, "w") do f
        write(f, str)
    end
    run(`gfortran -shared -fPIC -o $lib $file`)
    return lib
end

# Компилируем Fortran библиотеку
lib = @F raw"""
    integer(c_int) function factorial(n) bind(c, name="factorial")
        use, intrinsic :: iso_c_binding
        implicit none
        integer(c_int), intent(in), value :: n
        integer :: i

        factorial = 1
        do i = 1, n
            factorial = factorial * i
        end do
    end function factorial
"""

# Вызываем Fortran функции напрямую
n1 = 5
n2 = 3

# Вычисляем факториал
fact_result1 = ccall(:factorial, lib), Cint, (Cint,), n1)
fact_result2 = ccall(:factorial, lib), Cint, (Cint,), n2)

println("Число: ", n1)
println("Факториал: ", fact_result1)
println("Число: ", n2)
println("Факториал: ", fact_result2)
```

Число: 5

Факториал: 120

Число: 3

Факториал: 6

Теперь научимся передавать и возвращать массивы данных между кодом на C/Fortran и Julia

```
In [10]: macro C(str_expr)
    str = eval(str_expr)
    file = tempname() * ".c"
    lib = tempname() * ".so"

    open(file, "w") do f
        write(f, str)
    end
```

```

end

run(`gcc -shared -fPIC -o $lib $file`)
return lib
end

# Компилируем библиотеку
lib = @C raw"""
#include <stdlib.h>

double* add_arrays(double* arr1, double* arr2, int size) {
    double* result = (double*)malloc(size * sizeof(double));
    // Складываем массивы
    for (int i = 0; i < size; i++) {
        result[i] = arr1[i] + arr2[i];
    }
    return result; // Возвращаем указатель на новый массив
}
"""

# Тестируем
arr1 = [1.0, 2.0, 3.0, 4.0, 5.0]
arr2 = [10.0, 20.0, 30.0, 40.0, 50.0]

result_ptr = ccall((:add_arrays, lib), Ptr{Cdouble}, (Ptr{Cdouble}, Ptr{Cdouble}))
result_array = unsafe_wrap(Array{Cdouble}, result_ptr, length(arr1)) # unsafe
# Array{Cdouble} -тип создаваемого массива
# result_ptr - указатель на память, полученный из ccall

println("Первый массив: ", arr1)
println("Второй массив: ", arr2)
println("Результат сложения: ", result_array)
println("Тип результата: ", typeof(result_array))

```

Первый массив: [1.0, 2.0, 3.0, 4.0, 5.0]
 Второй массив: [10.0, 20.0, 30.0, 40.0, 50.0]
 Результат сложения: [11.0, 22.0, 33.0, 44.0, 55.0]
 Тип результата: Vector{Float64}

In [54]:

```

macro Fortran(str_expr)
    str = eval(str_expr)
    file = tempname() * ".f90"
    lib = tempname() * ".so"

    open(file, "w") do f
        write(f, str)
    end

    # Компилируем Fortran код в shared library
    run(`gfortran -shared -fPIC -o $lib $file`)
    return lib
end

# Компилируем Fortran библиотеку
lib_path = @Fortran raw"""
function add_arrays(arr1, arr2, size) result(result_ptr) bind(c, name="a

```

```

use, intrinsic :: iso_c_binding
implicit none
real(c_double), intent(in) :: arr1(*), arr2(*)
integer(c_int), intent(in), value :: size
type(c_ptr) :: result_ptr

real(c_double), pointer :: result_array(:)
integer :: i
allocate(result_array(size))

do i = 1, size
    result_array(i) = arr1(i) + arr2(i)
end do

result_ptr = c_loc(result_array)
end function add_arrays
"""

# Тестируем
arr1 = [1.0, 2.0, 3.0, 4.0, 5.0]
arr2 = [10.0, 20.0, 30.0, 40.0, 50.0]

result_ptr = ccall((:add_arrays, lib_path), Ptr{Cdouble}, (Ptr{Cdouble}, Ptr{Cdouble}), result_ptr, length(arr1))
result_array = unsafe_wrap(Array{Cdouble}, result_ptr, length(arr1))

println("Первый массив: ", arr1)
println("Второй массив: ", arr2)
println("Результат сложения: ", result_array)
println("Тип результата: ", typeof(result_array))

```

```

Первый массив: [1.0, 2.0, 3.0, 4.0, 5.0]
Второй массив: [10.0, 20.0, 30.0, 40.0, 50.0]
Результат сложения: [11.0, 22.0, 33.0, 44.0, 55.0]
Тип результата: Vector{Float64}

```

In []: